SEN418

Business Application Design and Development

BlogSpace

Frontend Blog Application

Student: Musa Halil Ecer

Student ID: B2105.090120

Course: SEN418 – Business Application Design and Development

Instructor: Assist. Prof. Dr. Raimon Bawazir

Submission: 26 May 2026

Work Type: Pairs (2 students per group)

Table of Contents

1. Introduction

BlogSpace is a modern, responsive frontend web application developed as a mini project for the SEN418 – Business Application Design and Development course. The application is a blog platform that allows users to browse, read, and create blog posts across a variety of categories including technology, travel, food, health, and more.

The application was built using React 19, Vite 7, and Tailwind CSS 4, following a component-driven architecture inspired by the Atomic Design methodology. The project demonstrates modern frontend development practices including client-side routing, lazy loading, responsive design, and reusable component composition.

1.1 Target Users

- General readers who want to discover and read blog articles
- Content creators and writers who want to publish blog posts
- Administrators who want to monitor blog statistics via the dashboard
- New users who want to register and join the platform

1.2 Core Services Provided

- Browse and read blog articles with author information and comments
- Create and publish new blog posts through an interactive form
- View platform statistics on the admin dashboard (total posts, comments, authors)
- Explore a travel image gallery
- User authentication UI (Login and Registration pages)

2. Requirements Analysis

2.1 Functional Requirements

#	Feature	Description	Priority
FR-01	**User Registration**	New users can create an account by providing email, username, and password	High
FR-02	**User Login**	Registered users can sign in with their credentials	High
FR-03	**View Blog List**	Users can browse all blog posts displayed as responsive cards on the home page	High
FR-04	**Read Blog Detail**	Users can click a card to read the full blog post along with comments	High
FR-05	**Create Blog Post**	Users can write and publish new blog posts via the Write Post form page	High
FR-06	**Dashboard Stats**	Admin can view total post count, comment count, author count, and reader stats	Medium
FR-07	**Image Gallery**	Users can view a travel photo gallery with a full-screen modal viewer	Medium
FR-08	**Navigation**	Users can navigate between all pages via the sidebar navigation menu	High
FR-09	**404 Handling**	Unknown routes display a user-friendly Not Found page	Low
FR-10	**Form Validation**	Write Post form validates required fields and shows error messages	Medium

2.2 Non-Functional Requirements

#	Category	Requirement	Priority
NFR-01	**Responsiveness**	Application must be fully responsive across mobile, tablet, and desktop screens	High
NFR-02	**Performance**	Pages should load within 2 seconds using Vite's build optimization and lazy loading	High
NFR-03	**Usability**	UI must follow modern design principles with clear navigation and readable typography	High
NFR-04	**Maintainability**	Code must follow Atomic Design pattern for easy component reuse and extension	Medium
NFR-05	**Compatibility**	Application must run on latest versions of Chrome, Firefox, and Edge	Medium
NFR-06	**Accessibility**	Sufficient color contrast and semantic HTML elements should be used	Medium

3. System Architecture

BlogSpace follows a Single Page Application (SPA) architecture, where all routing and rendering is handled on the client side. The application uses React Router DOM v7 with a createBrowserRouter setup, enabling nested layouts, lazy-loaded pages, and a shared Navbar component across all routes.

3.1 Architecture Style

- SPA (Single Page Application) — No page reloads; React manages the DOM
- Component-Driven Development — UI is composed from small reusable components
- Atomic Design — Components are organized as atoms → molecules → organisms → templates → pages
- Client-Side Routing — React Router DOM v7 handles navigation without backend requests
- Lazy Loading — Each page is loaded on demand using React.lazy and Suspense

3.2 Technology Stack

Layer	Technology	Version	Purpose
UI Framework	React	19.1.0	Component-based UI development
Build Tool	Vite	7.0.0	Fast dev server and optimized production builds
Styling	Tailwind CSS	4.1.11	Utility-first responsive styling
Routing	React Router DOM	7.6.3	Client-side navigation and nested layouts
Language	JavaScript (JSX)	ES2022+	Application logic and UI templates
Linting	ESLint	9.29.0	Code quality enforcement

3.3 Project Folder Structure

The project follows Atomic Design methodology with the following structure:

```
src/
components/
atoms/ → Button, Input, Image, Label
molecules/ → Card
organisms/ → Navbar, CardGroup
templates/ → Layout, Login, Register, NotFound
pages/ → home, about, dashboard, detail, images, writepost
data/ → blogs.js, comments.js (mock data)
model/ → blog.js, comment.js, user.js (data models)
router/ → route.jsx (all application routes)
main.jsx → Application entry point
index.css → Tailwind CSS imports
```

4. User Interface (UI/UX) Design

The application follows modern UI/UX principles with a clean, consistent design language using a blue and white color palette. All pages share the same Layout component which provides the sidebar navigation, ensuring a consistent user experience.

Login Page *(/login)*

Email/username + password form with 'Remember me' and 'Forgot password'. Clean card layout centered on the page. Links to the Register page.

Register Page *(/register)*

Email, username, password, and confirm password fields. Terms & Conditions checkbox. Fully validated form with modern card design.

Home Page *(/)*

Responsive grid of blog post cards (3 columns on desktop, 1 on mobile). Each card shows a cover image, title, description, date, and a 'Read More' button.

Dashboard *(/dashboard)*

4 statistics cards (Total Posts, Total Comments, Total Authors, Monthly Readers). Recent Posts data table with author avatars, dates, comment badges and action links. Top Posts chart with horizontal progress bars.

Write Post (Form Page) *(/write)*

Title, category dropdown, optional image URL with live preview, short summary, and full content textarea. Client-side form validation with error messages. Save Draft and Publish buttons. Success screen after submission.

Post Detail Page *(/detail/:id)*

Cover image, post title, author name, date and comment count. Full post content. Comments section with author avatars and message text.

Gallery Page *(/images)*

Grid of travel destination images with hover overlay effects showing titles. Click opens a full-screen modal with image and description.

About Page *(/about)*

Hero banner, mission statement, team member cards with profile images, and platform statistics (articles, readers, contributors, years).

Navigation Menu *(sidebar)*

Fixed sidebar on desktop (240px) with logo, active link highlighting, and collapsible hamburger menu on mobile. Groups: Main (Home, Dashboard, Write Post, Gallery, About) and Account (Login, Register).

404 Not Found *(*)*

Large 404 number, descriptive message, and navigation buttons back to Home or Dashboard.

5. Frontend Implementation

The frontend prototype is fully implemented and functional. All pages are accessible through the sidebar navigation, and the application runs locally with npm run dev. Below are the key implementation highlights.

5.1 Routing

React Router DOM v7 is used with createBrowserRouter. All routes are nested under a shared Layout component. Pages are loaded lazily with React.lazy and wrapped in Suspense with a loading spinner fallback, improving initial load performance.

5.2 Component Architecture (Atomic Design)

- Atoms — Button, Input, Image, Label: base-level UI primitives with configurable props
- Molecules — Card: combines Image, Button, and text to form a blog post card
- Organisms — Navbar, CardGroup: complex layout components built from atoms/molecules
- Templates — Layout, Login, Register, NotFound: page structure components
- Pages — home, dashboard, writepost, detail, images, about: full page views

5.3 State Management

The application uses React's built-in useState hook for local component state. The Write Post page manages form state and validation errors. The Gallery page manages the selected image for the modal. The Navbar manages the mobile menu toggle state. No external state management library is required for this prototype.

5.4 Responsive Design

Tailwind CSS utility classes are used for responsive breakpoints. On mobile screens, the sidebar is hidden and replaced with a fixed top bar and hamburger menu. Card grids adapt from 1 column (mobile) to 3 columns (desktop). All pages use responsive padding, font sizes, and layout adjustments.

5.5 Mock Data

Since this is a frontend-only prototype, data is stored in JavaScript files under src/data/. The blogs.js file contains 10 blog post objects with id, name, imageUrl, description, date, commentCount, authorName, and authorProfileUrl fields. The comments.js file contains 10 comment objects. Images are loaded from the Picsum Photos API for realistic placeholder content.

6. Database Design Concept

Although BlogSpace is a frontend-only prototype with no backend integration, a relational database schema has been designed to represent how data would be structured in a production environment. The design follows standard relational database principles. The complete ER Diagram is provided as a separate file.

6.1 Entities and Attributes

User
■ user_id (PK)
username
email
password_hash
profile_image_url
role
created_at

Blog
■ blog_id (PK)
title
description
content
image_url
■ author_id (FK → User)
■ category_id (FK → Category)
created_at
updated_at

Comment
■ comment_id (PK)
text
■ author_id (FK → User)
■ blog_id (FK → Blog)
created_at

Category
■ category_id (PK)
name
description

6.2 Relationships

- User — Blog: One-to-Many (one user can author many blog posts)
- Blog — Comment: One-to-Many (one blog post can have many comments)
- User — Comment: One-to-Many (one user can write many comments)
- Category — Blog: One-to-Many (one category can contain many blog posts)

7. Testing and Evaluation

7.1 Testing Methods

The application was tested manually throughout development. The following testing approaches were applied:

- Manual UI Testing — Each page was visually inspected on different screen sizes using Chrome DevTools responsive mode (mobile 375px, tablet 768px, desktop 1440px)
- Navigation Testing — All sidebar links were tested to ensure correct page routing. The active state highlighting was verified for each route.
- Form Validation Testing — The Write Post form was tested with empty fields to confirm that error messages appear correctly and prevent submission.
- Component Isolation Testing — Atom-level components (Button, Input, Image) were tested individually to ensure props are correctly applied.
- Error Route Testing — Navigating to an invalid URL was tested to confirm the 404 page renders.
- Cross-Browser Testing — Basic compatibility was verified in Chrome and Edge.

7.2 Known Limitations

- No backend integration — all data is mock/static and does not persist between sessions
- Authentication is UI-only — login and register forms do not connect to a real auth system
- Comments are not filtered per blog post — the detail page shows all comments regardless of post
- No search or filter functionality on the blog list

7.3 Possible Improvements

- Integrate a REST API or Firebase backend for real data persistence
- Add user authentication with JWT tokens or session cookies
- Implement comment submission directly from the detail page
- Add search and category filter on the home page
- Add pagination or infinite scroll for the blog list
- Write unit tests with Vitest and React Testing Library
- Add dark mode support using Tailwind's dark: variant

8. Conclusion

BlogSpace successfully demonstrates the core concepts of frontend business application development studied in SEN418. The application covers all required sections including a Login/Register page, Home page, Dashboard with statistics, a Write Post form, a data display page, and a consistent navigation menu.

The project was built using React 19, Vite 7, and Tailwind CSS 4, applying modern development practices such as Atomic Design, lazy loading, client-side routing, and responsive UI design. The component hierarchy is clean and maintainable, with clear separation between data, logic, and presentation layers.

Key lessons learned during this project include the importance of component reusability, the value of a well-structured routing system, and how Tailwind CSS's utility-first approach accelerates responsive layout development. The Atomic Design methodology proved effective for organizing components in a scalable way.

With a backend integration, real authentication, and a database, BlogSpace could serve as a fully functional blogging platform. The frontend prototype provides a solid and professional foundation for such an extension.